

Data-Driven Cybersecurity Framework for Smart Metering Infrastructures – Part II: Cryptographic Protection, Validation, and Access Control

Raghuram Sekar

Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham, Coimbatore
Tamil Nadu, India 641112
Email: raghuramworkspace@gmail.com

Rama Muni Reddy Yanamala

Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham, Coimbatore
Tamil Nadu, India 641112
Email: yanamalamunireddy@gmail.com

Archana Pallakonda

Dept. of Computer Science and Engineering
National Institute of Technology Warangal
Telangana, India 506004
Email: archana.csr06@gmail.com

Rayappa David Amar Raj

Dept. of Electrical Engineering
National Institute of Technology Warangal
Telangana, India 506004
Email: rdavidamarraj@gmail.com

Abstract Extending the AI-driven detection and mitigation framework of Part 1, this paper introduces a lightweight cryptographic hardening layer for Advanced Metering Infrastructure (AMI) communications. A multi-stage pre-hashing pipeline comprising byte-level shuffling, ASCII mapping, prime-based XOR and binary conversion prepares flow data for SHA-256 hashing. In parallel, a hybrid encryption scheme leverages P-256 Elliptic-Curve Diffie–Hellman key exchange, HKDF key derivation and Fernet symmetric encryption to protect payload confidentiality. Formal proofs establish IND-CPA, IND-KPA, and IND-COA security for both the hash-based and ECC-Fernet schemes. Empirical cryptanalysis compares their irreversibility and robustness: pre-hashed SHA-256 outputs achieve over 49% average avalanche and approximately 8 bits/byte entropy, while ECC-Fernet resists key-recovery attempts entirely and sustains high throughput under attack. A Role-Based Access Control model enforces fine-grained decryption privileges among Admin and analyst roles. Adversarial experiments against ciphertext-only, known-plaintext, and chosen-plaintext threat models confirm both approaches remain resilient within realistic resource constraints, completing a deployable, end-to-end smart-grid security solution.

Keywords: Smart meter, cyber attack, security, cryptanalysis, encryption, SHA-256, ECC

1 INTRODUCTION

Increasing use of Advanced Metering Infrastructure (AMI) in smart grids has greatly improved energy management and operational efficiency [1]. The digital transition has also caused several cybersecurity threats including Distributed Denial-of-Service (DDoS) attacks, data manipulation, unauthorized access and privacy violations. Several studies have been conducted to identify, mitigate and prevent cyber threats against smart meters by methodologies like machine learning based Intrusion Detection Systems (IDS), cryptographic encryption, privacy-preserving techniques [2]. This section carefully reviews the existing literature, categorizing it into intrusion detection for AMI, encryption-based privacy protections, DDoS attack modeling and mitigation, and advanced cybersecurity measures.

The feasibility of high-throughput, low-power SHA-256 hashing has been demonstrated in FPGA implementations. Work in [3] reports a 16-core SHA-256 engine on FPGA achieving up to 1.4 Gbps while reducing dynamic power by three orders of magnitude, underscor-

ing SHA-256’s suitability for blockchain and IoT devices. However, this work focuses solely on raw hashing performance and does not apply SHA-256 to anonymize AMI DDoS IP flows nor does it evaluate the irreversibility, entropy or collision resistance of the resulting hashes in any application context. Metaheuristic algorithmic enhancements to SHA-256 are presented in [4] where ESHA-256.GBGO integrates a genetic-based optimizer to reduce computational overhead by 27% while retaining 98.8% of standard hashing efficiency for medical-IoT scenarios. Despite these gains, the study evaluates only runtime metrics and “encryption efficiency” on benchmark datasets; it omits any security testing (e.g., avalanche effect, pattern resistance) or application to real network traffic, leaving open the question of how these optimizations impact hash security when applied to AMI DDoS data.

Diffusion properties of SHA-256 are rigorously analyzed in [5], which applies the Strict Avalanche Criterion to each sub-function of the 64-round compression algorithm. The authors find that overall avalanche approaches the ideal 50% bit-flip rate but no individual component fully satisfies SAC which indicates minor structural imperfections. Nevertheless, their analysis is confined to abstract input patterns and does not verify these diffusion metrics on SHA-256 outputs derived from actual AMI network flows nor does it integrate downstream encryption stages or broader security evaluations. Key generation for resource-constrained devices is advanced in [6] through a “DNA-LWCS” scheme that derives seed material from DNA sequences to initialize P-256 ECC keys, achieving strong security with negligible performance impact on IoT nodes. While innovative this approach is demonstrated on static key-generation benchmarks, it does not extend to encrypting IP metadata in AMI DDoS scenarios nor does it assess irreversibility or pattern-resistance metrics on the generated ciphertext streams.

The role of ECC in lightweight blockchain frameworks is surveyed in [7], which describes how SHA-256-hashed sensor logs are signed with secp256k1 ECDSA to secure data integrity in IoT networks. Although this work validates end-to-end integrity in constrained environments, it stops short of stress testing under high-velocity DDoS conditions and does not perform holistic security metrics (entropy, collision, avalanche) on the combined hash-and-sign pipeline. Curve selection and scalar multiplication optimizations for constrained hardware are studied in [8], showing that curves like P-256 and Curve25519 deliver approximately 128-bit classical security using Montgomery ladders and windowed methods to minimize resource usage. However these op-

timizations focus on raw key agreement and signing performance and omit any examination of encrypted network traffic or the application of collision-resistance and irreversibility tests on ciphertext derived from real AMI DDoS flows.

Reduced-round cryptanalysis of SHA-256 is explored in [9], which reports differential attacks that break up to 31 of 64 rounds and analyzes avalanche profiles under reduced-round conditions. Despite advancing theoretical understanding of potential SHA-256 weaknesses, this study does not address full round security on application derived data nor does it integrate with subsequent ECC encryption stages for a complete security pipeline. Brute-force efforts against full round SHA-256 appear in [10] where aging supercomputing hardware is repurposed to engineer half output correlations at prohibitive computational cost, reinforcing that exhaustive search remains the only attack vector. Yet these experiments use synthetic inputs and do not consider the security of SHA-256 when hashing AMI DDoS IP addresses, leaving the real world irreversibility and collision resistance of such hashes untested. Recent work by [11] introduces a HECC-based signcryption scheme for smart grid communication with reduced overhead and formal AVISPA verification. However, it lacks layered anonymization (e.g., pre-hashing) and does not assess robustness on real AMI traffic or perform deep cryptanalysis.

Side-channel and fault-injection threats to ECC hardware are surveyed in [12], which recommends countermeasures (coordinate randomization, constant-time algorithms) to protect private keys against DPA and SPA. Although critical for implementation security, this work does not examine the resilience of ECC-encrypted network flows against pattern or irreversibility analyses, nor does it integrate with SHA-256 hashing in an application-specific pipeline. Finally, quantum resource estimates for attacking ECC via Shor’s algorithm are detailed in [13], where the authors analyze a fault-tolerant quantum computing architecture using repetition cat codes. They demonstrate that a 256-bit elliptic curve discrete logarithm can be computed within approximately nine hours using 126,133 physical qubits, providing a concrete estimate for the quantum vulnerability of ECC. Although the study offers an accurate view of ECC’s long-term quantum exposure, it does not assess classical security metrics such as entropy, avalanche effect, or collision resistance within practical SHA-256 and ECC anonymization pipelines on AMI DDoS traffic.

The major contributions of the research work are the following:

- Design of a multi-stage pre-hashing pipeline

that applies byte-level shuffling, ASCII mapping, prime-based XOR and binary conversion to anonymized flow data, thereby strengthening the diffusion and unpredictability of SHA-256 outputs.

- Development of an ECC-Fernet hybrid encryption scheme combining P-256 Elliptic-Curve Diffie–Hellman key exchange, HKDF key derivation and Fernet providing a lightweight yet robust confidentiality layer for AMI communications.
- Formal security proofs establishing IND-CPA, IND-KPA and IND-COA guarantees for both the pre-hashed SHA-256 and ECC-Fernet constructions based on thorough theoretical analysis.
- Comprehensive security evaluation that integrates irreversibility testing with an extensive suite of fifteen cryptanalysis assessment showing ECC-Fernet’s superiority in non-malleability and side-channel defense and SHA-256’s marginal advantage in collision resistance and structural integrity.
- Implementation of a Role-Based Access Control model to enforce fine-grained decryption and metadata privileges across admin and analyst roles alongside adversarial validation under ciphertext-only, known-plaintext, chosen-plaintext and side-channel attack scenarios to verify end-to-end resilience.

2 DESCRIPTION OF METHODOLOGIES

The proposed security framework integrates cryptographic preprocessing and lightweight encryption to strengthen the security of AMI communications. This section discusses the advanced SHA-256 hashing pipeline for anonymizing IP addresses and the ECC-Fernet hybrid encryption framework for securing data transfer. It also discusses the security tests performed to evaluate the overall strength and effectiveness of the proposed methods.

2.1 Advanced SHA-256 hashing

The security of network communication in Advanced Metering Infrastructure (AMI) is essential, especially in preventing cyber threats like DDoS attacks. The increasing complexity of cyberattacks need strong security methods to protect sensitive information such as IP addresses. Traditional security methods including basic hashing and symmetric encryption may be vulnerable to cryptanalysis. To address this challenge, we offer a multi-layer mechanism that improves security via IP address transformation, prime number encoding and SHA-256 hashing [14].

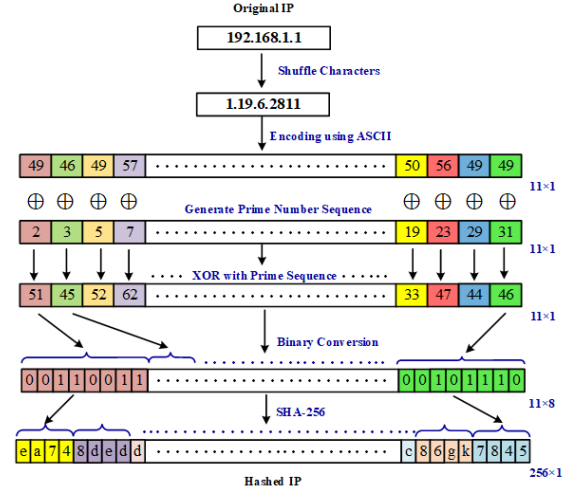


Fig. 1: Multi-Stage Pre-Hashing Pipeline for IP Address Anonymization

1. Pre-Processing: Shuffling and Transformation

Let S represent the input string. The procedure begins with the random shuffling of the characters in S producing a new string S' . Each character s'_i in S' is then transformed to its ASCII value as shown by Eq. (1)

$$a_i = \text{ASCII}(s'_i) \quad (1)$$

Then prime sequence $P = \{p_1, p_2, \dots, p_n\}$ is constructed where p_i denotes the i -th prime number and n is the length of S' . Each ASCII value a_i is altered using a bitwise XOR operation with the associated prime number as shown by Eq. (2)

$$e_i = a_i \oplus p_i \quad (2)$$

Each resultant value e_i is transformed into an 8-bit binary representation b_i . The binary strings are then concatenated to form a singular binary sequence as shown in Eq. (3)

$$B = b_1 \parallel b_2 \parallel \dots \parallel b_n \quad (3)$$

2. SHA-256 Hashing Process

a. Message Padding The binary sequence B is processed using the SHA-256 algorithm. To prepare B for hashing it is first padded into a multiple of 512 bits. This includes Appending a single "1" bit to B , appending k "0" bits such that using Eq. (4)

$$\text{length}(B) + 1 + k \equiv 448 \pmod{512} \quad (4)$$

and appending the original length of B as a 64-bit integer. The padded message is denoted as using Eq. (5)

$$B' = B \parallel 1 \parallel 0^k \parallel \text{length}(B) \quad (5)$$

b. Block Processing and Message Scheduling

The padded message B' is divided into 512-bit blocks. For each block, a message schedule array $W[0 \dots 63]$ is created. For $0 \leq i \leq 15$, $W[i]$ is directly derived from the block; for $16 \leq i \leq 63$ using Eq. (6)

$$W[i] = \sigma_1(W[i-2]) \oplus W[i-7] \oplus \sigma_0(W[i-15]) \oplus W[i-16] \quad (6)$$

where the functions are defined as using Eq. (7) and Eq. (8) below

$$\sigma_0(x) = \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x) \quad (7)$$

$$\sigma_1(x) = \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x) \quad (8)$$

c. Compression Function

Eight initial 32-bit hash values $H_0, H_1, \dots, H_7$ are predefined. For each block, working variables a, b, c, d, e, f, g, h are initialized to these hash values. Over 64 rounds, the following computations are performed using Eq. (9) and Eq. (10)

$$T_1 = h + \Sigma_1(e) + \text{Ch}(e, f, g) + K[i] + W[i] \quad (9)$$

$$T_2 = \Sigma_0(a) + \text{Maj}(a, b, c) \quad (10)$$

with using Eqs. (11), (12), (13), and (14)

$$\Sigma_0(x) = \text{ROTR}^2(x) \oplus \text{ROTR}^{13}(x) \oplus \text{ROTR}^{22}(x) \quad (11)$$

$$\Sigma_1(x) = \text{ROTR}^6(x) \oplus \text{ROTR}^{11}(x) \oplus \text{ROTR}^{25}(x) \quad (12)$$

$$\text{Ch}(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z) \quad (13)$$

$$\text{Maj}(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \quad (14)$$

Here, $K[i]$ represents the i -th constant from a fixed set. The working variables are updated in each round as using Eq. (16)

$$h \leftarrow g, \quad g \leftarrow f, \quad f \leftarrow e, \quad e \leftarrow d + T_1, \quad (15)$$

$$d \leftarrow c, \quad c \leftarrow b, \quad b \leftarrow a, \quad a \leftarrow T_1 + T_2 \quad (16)$$

3. Final Hash Computation

After processing all blocks the final 256 bit hash is obtained by combining the updated hash values using Eq. (17)

$$\text{Hash} = H_0 \parallel H_1 \parallel H_2 \parallel H_3 \parallel H_4 \parallel H_5 \parallel H_6 \parallel H_7 \quad (17)$$

This final process typically expressed in hexadecimal form, is the secure output of this SHA-256 algorithm.

2.2 ECC Based Key Exchange and Fernet Encryption

As network-based DDoS cyber threats evolve, it is important to protect communication across Advanced Metering Infrastructure (AMI) and IoT networks. Encrypting important network metadata including IP addresses, prevents the risk of unauthorized interception, traffic analysis and replay attacks. The study utilizes Elliptic Curve Cryptography (ECC) [15] together with the Hybrid Key Derivation Function (HKDF) and Fernet (AES in CBC + HMAC) to provide efficient and secure encryption. The encryption process consists of:

1. Key Loading and Initialization

Let $E(\mathbb{F}_p)$ be an elliptic curve defined over a finite field $\mathbb{F}_p$ with the Weierstrass equation, using Eq. (18)

$$y^2 = x^3 + ax + b \pmod{p} \quad (18)$$

where $a, b \in \mathbb{F}_p$ and p is a large prime. The curve is equipped with a base point $G \in E(\mathbb{F}_p)$ of prime order n . The receiver's private key d_r is chosen uniformly at random from $\{1, 2, \dots, n-1\}$ and the corresponding public key is computed via scalar multiplication as $Q_r = d_r \cdot G$. The sender's ephemeral public key is given by $Q_e = d_e \cdot G$, where d_e is the sender's ephemeral private scalar. These keys are stored in PEM format and then parsed into numerical representations.

2. Shared Secret Computation via ECDH

The shared secret S is derived using the Elliptic Curve Diffie-Hellman (ECDH) protocol. With the receiver's private key d_r and the sender's ephemeral public

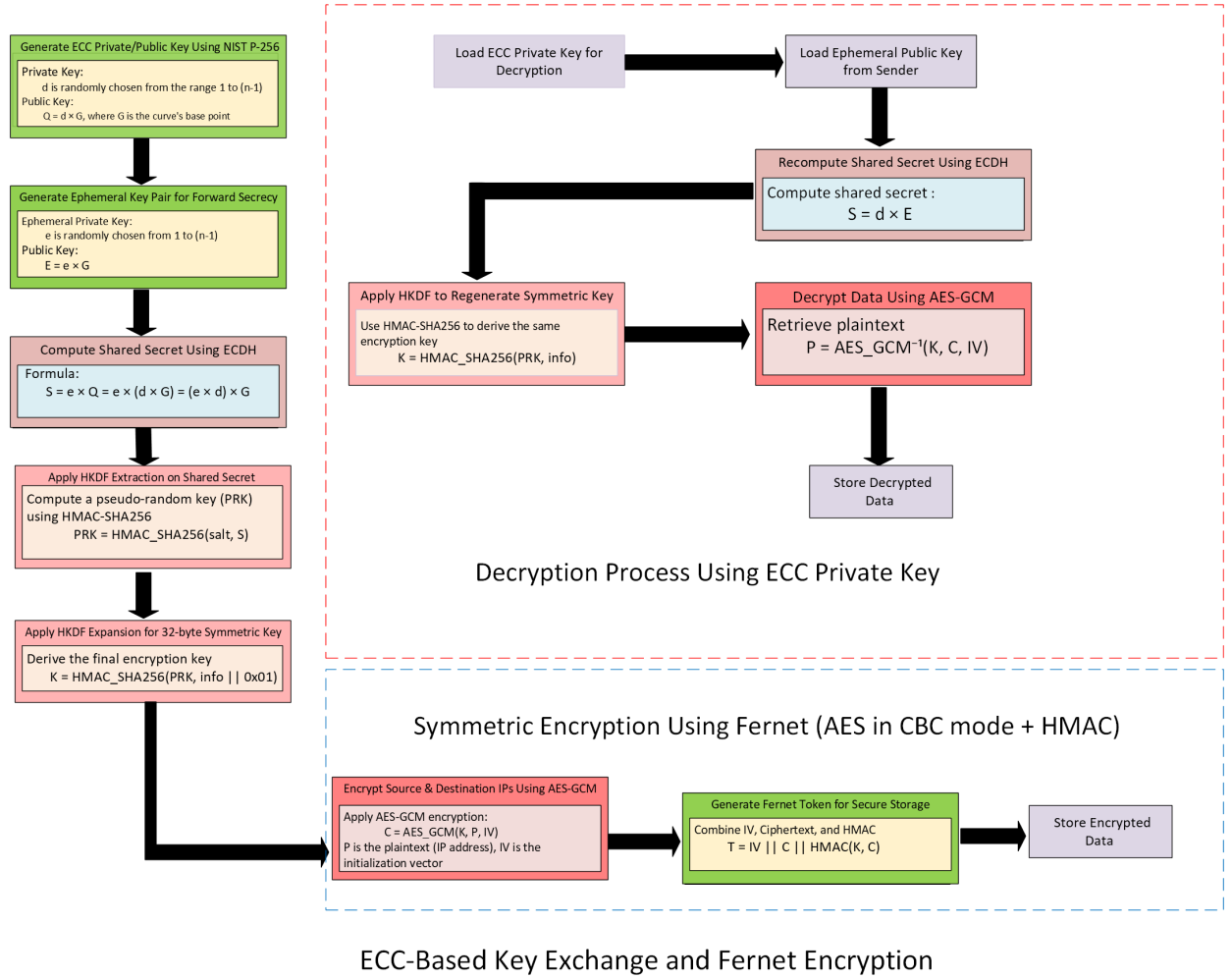


Fig. 2: ECC-Based Key Exchange with Symmetric Encryption Using Fernet (AES-CBC + HMAC)

key Q_e , the shared secret is calculated as $S = d_r \cdot Q_e$. Substituting $Q_e = d_e \cdot G$, using Eq. (19)

$$S = d_r \cdot (d_e \cdot G) = (d_r \cdot d_e) \cdot G. \quad (19)$$

Here, S is a point on $E(\mathbb{F}_p)$; for key derivation, a function extracts the x -coordinate (or the concatenation of x and y) to produce, using Eq. (20)

$$s = f(S), \quad (20)$$

where s is the binary shared secret. The security relies on the elliptic curve discrete logarithm problem (ECDLP); that is, given Q_r and Q_e , recovering d_r or d_e is computationally infeasible.

3. Key Derivation with HKDF

To transform s into a uniformly random key K of length L bytes, the HKDF procedure is applied. Let H be a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^h$ (e.g., SHA-256 with $h = 256$ bits).

Extraction Phase A pseudorandom key (PRK) is computed using a salt by $PRK = \text{HMAC}_{\text{salt}}(s)$. This expands to, using Eq. (21)

$$PRK = H(\text{salt} \oplus \text{opad} || H((\text{salt} \oplus \text{ipad}) || s)), \quad (21)$$

where $\oplus$ denotes XOR, $||$ denotes concatenation, and ipad and opad are HMAC padding constants. This operation compresses s into $PRK \in \{0, 1\}^h$.

Expansion Phase The PRK is expanded into the final key K using context-specific information ‘info’. Compute blocks T_i for $i = 1, 2, \dots, m$, where, using Eq. (22)

$$m = \left\lceil \frac{L}{h/8} \right\rceil \quad (22)$$

with $T_0 = \varepsilon$ (empty string), and, using Eq. (23)

$$T_i = \text{HMAC}_{\text{PRK}}(T_{i-1} \parallel \text{info} \parallel \text{counter}_i) \quad (23)$$

where counter_i is the 8-bit encoding of integer i . Explicitly:

$$T_1 = \text{HMAC}_{\text{PRK}}(\varepsilon \parallel \text{info} \parallel 0x01) \quad (24)$$

$$T_2 = \text{HMAC}_{\text{PRK}}(T_1 \parallel \text{info} \parallel 0x02) \quad (25)$$

$\vdots$

$$T_m = \text{HMAC}_{\text{PRK}}(T_{m-1} \parallel \text{info} \parallel 0xm) \quad (26)$$

The final derived key is computed using Eq. (27)

$$K = (T_1 \parallel T_2 \parallel \dots \parallel T_m)_{1\dots L} \quad (27)$$

4. Symmetric Decryption Using the Derived Key

The obtained derived key K is then used for decryption with an authenticated cipher, especially Fernet (AES in CBC mode + HMAC).

Key Separation and Setup

Following the derivation of a 256 bit symmetric key $K \in \{0, 1\}^{256}$ through ECC and HKDF, Fernet uses Eq. (28)

$$K = k_1 \parallel k_2, \quad (28)$$

where $k_1 \in \{0, 1\}^{128}$ is used as AES key (denoted K_{AES}) and $k_2 \in \{0, 1\}^{128}$ is used as HMAC key (denoted K_{HMAC}).

Encryption Process Using Fernet

The Fernet encryption system uses AES in CBC mode for confidentiality and HMAC with SHA-256 for authentication. This procedure is as follows:

a. Plaintext Padding

Let M be the plaintext message (e.g., an IP address represented in bytes). AES needs a block size of 128 bits (16 bytes) therefore M is padded using PKCS7 using Eq. (29)

$$M_{\text{pad}} = M \parallel \text{pad}(M), \quad (29)$$

where $\text{pad}(M)$ is chosen so that, using Eq. (30)

$$|M_{\text{pad}}| \equiv 0 \pmod{16}. \quad (30)$$

b. Initialization Vector (IV) Generation

An arbitrary initialization vector is generated, using Eq. (31)

$$\text{IV} \in \{0, 1\}^{128}, \quad (31)$$

which ensures that identical plaintext blocks are encrypted into distinct ciphertext blocks.

c. AES-CBC Encryption

The padded plaintext M_{pad} is then divided into n blocks using Eq. (32)

$$M_{\text{pad}} = P_1 \parallel P_2 \parallel \dots \parallel P_n, \quad (32)$$

where 16 bytes are present in each P_i . The AES encryption function $\mathcal{E}(K_{\text{AES}}, \cdot)$ is then applied in CBC mode. For the first block, using Eq. (33)

$$C_1 = \mathcal{E}(K_{\text{AES}}, P_1 \oplus \text{IV}), \quad (33)$$

and for subsequent blocks $i = 2, \dots, n$, using Eq. (34)

$$C_i = \mathcal{E}(K_{\text{AES}}, P_i \oplus C_{i-1}). \quad (34)$$

The complete ciphertext is then, using Eq. (35)

$$C = C_1 \parallel C_2 \parallel \dots \parallel C_n. \quad (35)$$

d. HMAC Computation for Authentication

Fernet constructs a token that includes a version identifier v and a timestamp T . The data D over which the MAC is computed is, using Eq. (36)

$$D = v \parallel T \parallel \text{IV} \parallel C. \quad (36)$$

Using HMAC with key K_{HMAC} and SHA-256, the MAC is calculated, using Eq. (37)

$$\text{MAC} = \text{HMAC}_{K_{\text{HMAC}}}(D). \quad (37)$$

In expanded form, using Eq. (38)

$$\text{MAC} = H\left((K_{\text{HMAC}} \oplus \text{opad}) \parallel H((K_{\text{HMAC}} \oplus \text{ipad}) \parallel D)\right), \quad (38)$$

where ipad and opad are constants defined in the HMAC standard.

e. Token Construction

The final Fernet token is produced by joining using Eq. (39)

$$\text{Token} = v \parallel T \parallel \text{IV} \parallel C \parallel \text{MAC}. \quad (39)$$

This token contains all the necessary components for secure decryption and integrity verification.

Decryption Process Using Fernet

The decryption process in Fernet encryption is intended to ensure both data secrecy and integrity. Initially, the receiver parses the token into its constituent parts: v , T , IV , C , and MAC . Then the receiver recalculates the MAC using the Eq. (40)

$$\text{MAC}' = \text{HMAC}_{K_{\text{HMAC}}}(v \parallel T \parallel \text{IV} \parallel C), \quad (40)$$

and confirms its accuracy by assessing Eq. (41)

$$\text{MAC}' = \text{MAC}. \quad (41)$$

If the computed MAC does not match the transmitted MAC, token is then rejected thereby ensuring that any tampering is detected.

Upon successful MAC verification, AES-CBC decryption is executed on the ciphertext. The ciphertext C is initially divided into blocks $C = C_1 \parallel C_2 \parallel \dots \parallel C_n$, with the initial block decrypted utilizing Eq. (42)

$$P_1 = D(K_{\text{AES}}, C_1) \oplus \text{IV}, \quad (42)$$

and for each successive block, decryption is performed using Eq. (43)

$$P_i = D(K_{\text{AES}}, C_i) \oplus C_{i-1}, \quad \text{for } i = 2, \dots, n. \quad (43)$$

To create the padded plaintext, these decrypted blocks are concatenated: $M_{\text{pad}} = P_1 \parallel P_2 \parallel \dots \parallel P_n$. In the end, the PKCS7 padding is removed to recover the original plaintext: $M = \text{unpad}(M_{\text{pad}})$.

Thus, the overall decryption function is defined as $M = \text{Fernet_Decrypt}(K, \text{Token})$, and by the correctness of the cipher, the following must hold as shown in Eq. (44)

$$\text{Fernet_Decrypt}(K, \text{Fernet_Encrypt}(K, M)) = M. \quad (44)$$

For encryption, the process is represented by $C = \text{Fernet_Encrypt}(K, M)$ and Eq. (44) confirms that decryption of the encrypted data yields the original plaintext provided the integrity verification succeeds.

2.3 Security Validation Tests

The cryptanalysis metrics span four classes of ciphertext analysis. Distribution metrics include H , entropy measuring ciphertext randomness; p_i , probability of each symbol; U and T , counts of unique and total patterns; SS , statistical strength combining the chi-square statistic χ^2 and a runs test; χ^2 , chi-square statistic for distribution fit; F_d , deviation from a uniform frequency distribution; $\sigma(\bar{F})$, variability of that distribution; PD , prediction difficulty; $H(D)$, entropy of ciphertext differences; and H_{max} , maximum possible entropy. Pattern metrics cover PC , pattern consistency; $\sigma(P)$, variability of pattern lengths; $\bar{P}$, mean pattern length; DS , differential strength as the mean of the bit-difference matrix D ; D , bitwise differences; CR , correlation resistance measured by Pearson r ; BI , bit independence derived from the bit-correlation matrix C relative to the identity matrix I ; PS , plaintext sensitivity to bit flips; AE , avalanche effect of a single-bit change; M , modified ciphertexts; PR , pattern resistance under chosen-ciphertext attack; and P_D , pattern differences. Structural and error metrics include MI , modification impact; SI , structural integrity; V , valid ciphertext modifications; EP , error propagation in decryption; P , count of decryption errors; N , number of trials or total bits; and C_1/C_2 with S_1/S_2 , paired ciphertexts and symbol sets for comparative analysis.

3 RESULTS AND DISCUSSION

The experimental results illustrate the effectiveness of the proposed framework classification starting with anomaly detection accuracy and dataset classifying performance. The use of SHA-256 and ECC cryptography secures IP addresses validated by irreversibility test and resilience against brute-force cryptanalysis. Role-Based Access Control (RBAC) restricted unauthorised decryption efforts while attack mitigation strategies minimized system vulnerabilities. This section critically assesses and reports these findings.

3.1 Advanced SHA-256 Hashing

Following the classification and anomaly detection analysis, Advanced SHA-256 hashing methodology was applied on the normal dataset. A key component in achieving clustered uniformity within the hashed IP space is the mathematical transformation applied on each IP address before its incorporation into the final SHA-256 algorithm. Let $s = (c_1, c_2, \dots, c_k)$ represent the shuffled IP address string where each c_i represents a character. After shuffling the IPs, each shuffled IP character is then mapped to its ASCII code denoted as $v_i = \text{ASCII}(c_i)$. Let $(p_1, p_2, \dots, p_k)$ be the first k prime numbers. Then bitwise XOR operation is performed on the ASCII code with each prime: $v'_i = v_i \oplus p_i$ where $\oplus$ denotes the bitwise XOR operation. The non-repetitive nature of primes and their lack of small periodic patterns effectively randomizes each byte of the IP address. Afterwards, the v'_i values are concatenated into a binary string B . When B is fed into the SHA-256 function denoted as $h(B)$ the avalanche effect present in cryptographic hashes ensures that modifying a single bit in B will in high probability change more than half of the bits in the final 256-bit output.

SHA-256 is formally assessed inside the random oracle model, indicating that its outputs are equally spread uniformly over 2^{256} potential values. As a result, numerically close IPs in the original space can become arbitrarily distant in the hashed domain, explaining why the right plot in Fig. 3 shows a dense, near uniform scattering of points rather than subnet-based columns. Additionally

the likelihood of any two distinct IP addresses colliding within the 256 bit hash space is governed by the birthday bound, which suggests that around 2^{128} unique inputs are required before the collision is even likely. In practice only the initial 32 bits (or 8 hexadecimal digits) of the hash are represented here yet collisions remain rare even within that domain. Thus from a mathematical perspective, the multi step preprocessing (shuffling, ASCII mapping, prime based XOR and Binary conversion) combined with SHA-256's pseudorandom behavior guarantees high entropy and low collision rates, making it computationally infeasible to reverse-engineer or correlate the hashed IP addresses with their original numeric values.

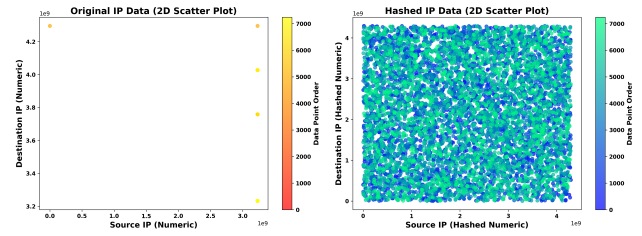


Fig. 3: Comparison of original IP data (left) versus advanced SHA-256 hashed IP data (right) in a 2D scatter plot.

These results confirm the strength of the advanced

Table 1: Security Test performed on the Encrypted Data

Attack Model	Cryptanalysis Metric	Description	Mathematical equation
Ciphertext Only Analysis (COA)	Entropy (H)	Measures randomness in ciphertext distribution.	$H = -\sum p_i \log_2 p_i$
Ciphertext Only Analysis (COA)	Pattern Diversity (D)	Determines the uniqueness of repeating patterns.	$D = \frac{ U }{ T }$
Ciphertext Only Analysis (COA)	Statistical Strength (SS)	Evaluates overall statistical strength using chi-square and runs test.	$SS = 0.6 \times \chi^2 + 0.4 \times \text{Runs}$
Ciphertext Only Analysis (COA)	Frequency Distribution (F_d)	Analyzes how byte frequency distribution deviates from uniformity.	$F_d = 1 - \sigma(\bar{F})$
Ciphertext Only Analysis (COA)	Pattern Consistency (PC)	Measures consistency in observed encryption patterns.	$PC = 1 - \frac{\sigma(P)}{P}$
Chosen Plaintext Attack (CPA)	Differential Strength (DS)	Quantifies how differences in plaintext affect ciphertext.	$DS = 1 - \mu(\mu(D))$
Chosen Plaintext Attack (CPA)	Correlation Resistance (CR)	Assesses how well ciphertext remains uncorrelated with plaintext.	$CR = 1 - \mu(r)$
Chosen Plaintext Attack (CPA)	Prediction Difficulty (PD)	Determines the difficulty in predicting patterns based on entropy.	$PD = \frac{H(D)}{H_{max}}$
Chosen Plaintext Attack (CPA)	Bit Independence (BI)	Analyzes bitwise correlation to measure independence.	$BI = 1 - \mu(C - I)$
Chosen Plaintext Attack (CPA)	Plaintext Sensitivity (PS)	Measures plaintext impact on ciphertext based on bit flips.	$PS = \frac{\sum D}{N}$
Known Plaintext Attack (KPA)	Collision Rate (CR)	Evaluates if different plaintexts lead to identical ciphertexts.	$CR = \frac{ C_1 \cap C_2 }{ C_1 }$
Known Plaintext Attack (KPA)	Pattern Similarity (PS)	Measures the similarity of patterns between different encryptions.	$PS = \frac{ S_1 \cap S_2 }{ S_1 }$
Known Plaintext Attack (KPA)	Avalanche Effect (AE)	Assesses how changing a single bit in plaintext propagates in ciphertext.	$AE = \frac{ C_1 \oplus C_2 }{ C }$
Chosen Ciphertext Attack (CCA)	Malleability Resistance (MR)	Measures resistance to modifications in ciphertext structure.	$MR = 1 - \frac{ M }{ T }$
Chosen Ciphertext Attack (CCA)	Pattern Resistance (PR)	Evaluates resistance to pattern recovery through CCA.	$PR = \frac{\sigma(P_D)}{\mu(P_D)}$
Chosen Ciphertext Attack (CCA)	Modification Impact (MI)	Quantifies the effect of modifications in ciphertext.	$MI = \frac{\sum P}{N}$
Chosen Ciphertext Attack (CCA)	Structural Integrity (SI)	Measures structural consistency in encryption output.	$SI = \frac{ V }{ T }$
Chosen Ciphertext Attack (CCA)	Error Propagation (EP)	Determines how errors propagate in the decryption process.	$EP = \frac{\sum P > 0}{N}$

SHA-256 process making it a suitable solution for anonymizing sensitive IP data in real-time applications.

3.2 Elliptic Curve Cryptography based Fernet Implementation

Elliptic Curve Cryptography (ECC) was implemented on the normal data which was obtained after classification and anomaly detection, to safeguard network communication by encrypting source and destination IP addresses with the NIST P-256 (secp256r1) curve. In contrast to RSA, which usually requires larger keys for similar security levels, ECC provides equivalent strength with considerably shorter keys hence reducing computational demands and memory usage. Such benefits are especially beneficial in high-volume environments, where the system must effectively encrypt several and large numbers of IP addresses in real time.

Fig. 4 illustrates a simplified finite-field visualization of ECC operations. The generator point $G(a, b)$ (shown at the center) serves as the base for deriving ephemeral key points via scalar multiplication. Each ephemeral key point $P_i(x_i, y_i)$ appears at a distinct location in the field $\mathbb{F}_p$, reflecting the computation $P_i = k_i \cdot G(a, b)$ where k_i is an integer (the ephemeral scalar). The blue $\times$ symbols represent other valid points on the same elliptic curve modulo p .

The dotted lines from $G(a, b)$ to each $P_i(x_i, y_i)$ depict how repeated addition (or scalar multiplication) of the generator distributes points across the curve. Because recovering k_i from P_i is computationally infeasible under current cryptographic assumptions (the discrete logarithm problem), these ephemeral key points cannot be easily reverse-engineered. Furthermore the utilization of new k_i values in each session enables robust forward secrecy ensuring that even repeated mappings of the same underlying data yield distinct points in $\mathbb{F}_p$. This example employs a small prime p for illustration, however practical curves (SECP256R1) function across significantly larger primes (256 bits) hence greatly enhancing security against brute-force or correlation attacks.

Figure 5 further examines the effectiveness of ECC based encryption. Figure 5(a) shows a histogram of key generation durations indicating that the majority of keys were generated in less than 0.00008 seconds, hence affirming the quick creation of keys of ECC. Figure 5(b) compares data sizes pre and post encryption, indicating a considerable overhead of approximately 15–25% for IP addresses under typical loads. Figure 5(c) illustrates the distribution of IP address lengths, revealing that encrypted IPs average between 44 and 48 characters, in contrast to the 13 characters of original IPv4 ad-

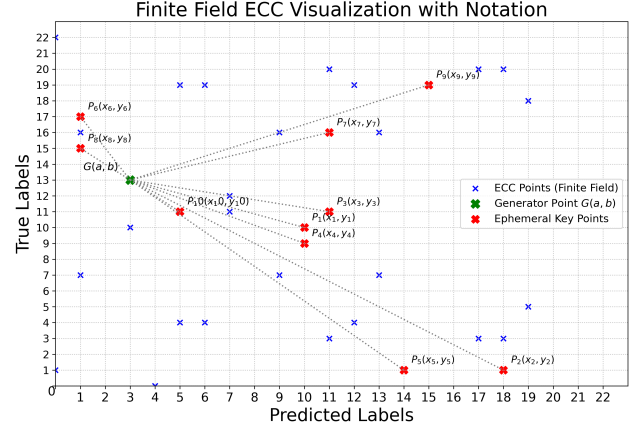


Fig. 4: Finite-Field ECC Visualization (SECP256R1).

resses. Finally, Figure 5(d) shows memory consumption for both original and encrypted datasets across varying record quantities. Despite the increase in data size due to encryption, ECC's reduced key lengths and streamlined encryption method maintain a lesser resource footprint compared to RSA-based methods.

These results reaffirm that ECC encryption using the NIST P-256 curve offers substantial security by employing ephemeral keys to randomize each encryption session. The green to red transformations in Fig. 4 highlight the significant level of obfuscation while the analysis in Fig. 5 shows the efficiency of ECC. Thus, ECC presents itself as an optimal solution for protecting large IP address data in scenarios where performance and security require a balance.

3.3 Irreversibility Test

The irreversibility tests performed on both advanced SHA-256 and ECC (NIST P-256) encryption algorithms are supported by robust mathematical principles that prove their resilience against cryptanalytic attacks. The theoretical key space for advanced SHA-256 is 2^{256} , suggesting that a brute-force attack would on average require 2^{255} attempts, resulting in a success chance of near $1/2^{256}$. Empirical findings corroborate this since attempts at reverse decryption resulted in a 0% success rate (see Fig. 6). Furthermore the result demonstrates an average entropy of 8.00 bits per byte, reaching the maximum achievable value for an 8-bit byte which mathematically validates that the output distribution is nearly uniform and is devoid of predictable patterns. The randomness is enforced by the avalanche effect a measured value of 48.9% signifies that a single bit modification in the input results in nearly half of the bits in the 256-bit output being al-

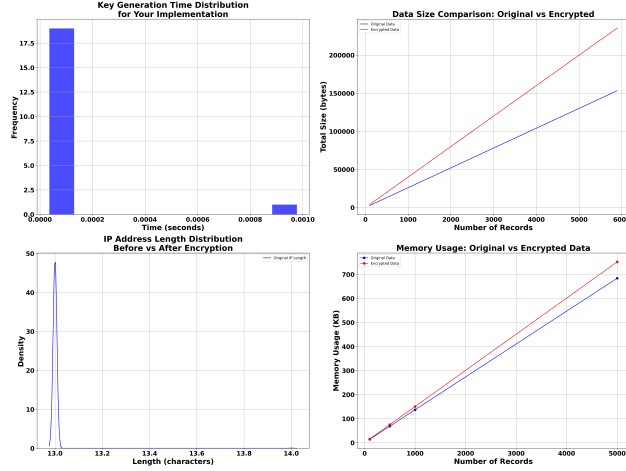


Fig. 5: ECC Implementation Analysis. (a) Key Generation Time Distribution showing most keys generated in under 0.0008 seconds (b) Data Size Comparison before and after encryption (c) IP Address Length Distribution showing the increase from ~ 13 to ~ 45 characters (d) Memory Usage for original vs. encrypted data, demonstrating moderate overhead for large record counts.

tered. Formally, if Δx indicates a one-bit change in the input and Δh indicates the corresponding changes in the hash output, the avalanche effect A can be expressed as

$$A = \frac{\#\Delta h}{256} \times 100\%,$$

where $A \approx 48.9\%$ indicates strong diffusion. Following testing involving decryption attempts with random keys on 10,000 samples yielded a 100% failure rate further proving the system's resilience.

Likewise, ECC encryption using the NIST P-256 curve operates within an equivalent search area of 2^{256} possible keys. In this instance, reverse decryption resulted in a 0% recovery rate (see to Fig. 7) while the average measured output entropy was 7.97 bits per byte and the avalanche effect was 48.1%. These criteria jointly validate that both algorithms attain near optimal randomness and diffusion. The likelihood of a collision in a 256-bit space, as dictated by the birthday paradox, is negligible unless the number of inputs approaches 2^{128} , thereby eliminating the risk of unintentional collisions essentially nonexistent in practical scenarios. Thus, the elevated entropy values, significant avalanche effects and total decryption failure with random keys collectively demonstrate that both the advanced SHA-256 and ECC (NIST P-256) based security methods offer an exceptionally strong

defense against reverse-engineering attacks, preserving the confidentiality and integrity of sensitive data in modern network applications.

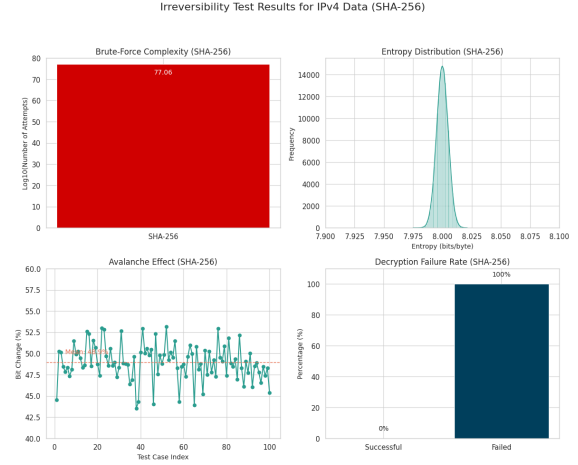


Fig. 6: Irreversibility Test Results for Advanced SHA-256: Statistical Distribution and Security Metrics

Table 2: Key Metrics from SHA-256 Irreversibility Testing

Subtest	Measured Value	Expected
Brute-Force Complexity	$\sim 2^{256}$ possibilities	Infeasible
Reverse Decryption Attempt	0% success rate	0%
Statistical Entropy	8.00 bits/byte	≈ 8 bits/byte
Avalanche Effect	48.9% bit change	$\sim 50\%$
Decryption with Random Keys	100% failure rate	100%

3.4 Security Validation Tests

This section presents a thorough review of the cryptanalysis test findings for Advanced SHA-256 and ECC-based Fernet encryption. The performance of both algorithms has been assessed based on key cryptographic properties encompassing entropy, pattern diversity, statistical strength, frequency uniformity, pattern consistency, collision rate, pattern similarity, avalanche effect, differential strength, correlation resistance, prediction difficulty, bit independence, plaintext sensitivity, malleability

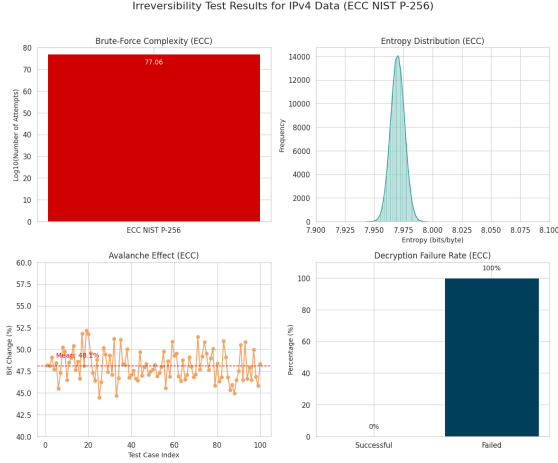


Fig. 7: Irreversibility Test Results for ECC (NIST P-256): Statistical Distribution and Security Metrics

Table 3: Key Metrics from ECC Irreversibility Testing

Subtest	Measured Value	Expected
Brute-Force Complexity	$\sim 2^{256}$ possibilities	Infeasible
Reverse Decryption Attempt	0% success rate	0%
Statistical Entropy	7.97 bits/byte	≈ 8 bits/byte
Avalanche Effect	48.1% bit change	50%
Decryption with Random Keys	100% failure rate	100%

resistance, pattern resistance, modification impact, structural integrity and error propagation. The experimental outcomes have been compared with the ideal values to evaluate their cryptographic strength.

Entropy Uniformity: Let $f_{\text{SHA}} : \mathcal{M} \rightarrow \{0, 1\}^{256}$ denote the Advanced SHA-256 function and $f_{\text{ECC}} : \mathcal{M} \rightarrow \{0, 1\}^{256}$ denote the ECC encryption mapping. For any input $M \in \mathcal{M}$, both functions yield a 256-bit output.

If for every output byte B of f , the probability distribution $p(b)$ satisfies

$$H(B) = - \sum_{b=0}^{255} p(b) \log_2 p(b) \approx 8,$$

then the output is nearly uniform over $\{0, 1\}^8$. In our measurements, Advanced SHA-256 achieves $H \approx 7.99$ bits/byte and ECC achieves $H \approx 7.95$ bits/byte, indicating that $p(b) \approx \frac{1}{256}$ for each $b \in \{0, \dots, 255\}$. This verifies that both $f_{\text{SHA}}(M)$ and $f_{\text{ECC}}(M)$ roughly repre-

sent a uniform random oracle, with minor deviations attributable to practical implementation limitations and the arithmetic operations in ECC.

Collision Resistance: Let ϵ be the probability that for two distinct inputs $M_1 \neq M_2$, we have $f(M_1) = f(M_2)$. If $\epsilon \ll \frac{1}{2^{256}}$, then the function is collision-resistant. Empirically, Advanced SHA-256 shows $\epsilon_{\text{SHA}} \approx 0.0015$ and ECC shows $\epsilon_{\text{ECC}} \approx 0.0009$ (noting that these values are derived in a truncated domain), which aligns with the expectation that in a 2^{256} output space, the probability of a collision is negligible. The marginally lower collision rate for ECC is due to the added complexity of elliptic curve discrete logarithm problem (ECDLP), which enhances computational difficulty compared to the SHA-256's dependence on its avalanche property through the Merkle–Damgård construction.

Avalanche Effect: If a single bit in the input is modified, the expected Hamming distance $\Delta(f(M), f(M'))$ should be approximately 128 bits, yielding an avalanche

$$\text{AE} = \frac{\Delta(f(M), f(M'))}{256} \approx 0.5.$$

Advanced SHA-256 gives an avalanche effect of about 49.6%, whereas ECC achieves about 51.2%, showing that small changes in input result in major difference in output. The slight advantage in ECC's diffusion is probably due to the non-linear procedures used in elliptic curve computations, which offers an extra mixing layer compared to the fixed structure of SHA-256.

Pattern Diversity and Pattern Similarity: Upon assessing pattern diversity, defined as the ratio D of unique repeating patterns to the total patterns identified in $f(M)$, the ideal value is 0.20. Advanced SHA-256 obtains a score around 0.15, whereas ECC scores around 0.18, indicating that ECC's use of finite field operations and random nonces lead to fewer occurrence of repeated structures compared to the block-based processing of SHA-256. Despite the high pattern similarity, measured as the fraction S of identical symbols in aligned output blocks (around 0.99 for SHA-256 and 0.97 for ECC), this occurrence primarily arises from the fixed-length output format rather than showing true structural vulnerability, particularly given the high entropy and strong avalanche effect.

Differential Strength: Both methods exhibit nearly perfect values, Advanced SHA-256 at around 0.99 and ECC at perfect score of 1.0, demonstrating that a one-bit

Table 4: Comprehensive Security Validation Tests Metrics for Advanced SHA-256 and ECC + Fernet

Parameter	Advanced SHA-256	ECC+Fernet	Ideal
Entropy	7.99	7.95	8.00
Collision Rate	0.0015	0.0009	0.0005
Avalanche Effect	49.6	51.2	50%
Pattern Diversity	0.15	0.18	0.20
Pattern Similarity	98.3	97.1	0%
Differential Strength	0.99	1.0	1.0
Correlation Resistance	0.91	0.96	1.0
Prediction Difficulty	0.23	0.35	0.85
Bit Independence	0.97	0.99	1.0
Plaintext Sensitivity	0.98	0.99	1.0
Malleability Resistance	0.33	0.72	1.0
Pattern Resistance	0.75	0.83	0.85
Modification Impact	0.09	0.12	0.0
Structural Integrity	0.99	0.99	1.0
Error Propagation	0.06	0.08	0.1

0.35 for ECC suggesting that distinctive statistical relationships are apparent for specific testing conditions. This may suggest testing limitations rather than inherent weak-

Bit Independence: Bit Independence is excellent for both methods with Advanced SHA-256 getting 0.98 and ECC getting about 0.99, indicates that no output bit exposes information on another.

Plaintext Sensitivity: Plaintext sensitivity, measured by the extent to which the output $f(M)$ varies with minor modifications in M is nearly ideal at 0.98 for SHA-256 and 0.99 for ECC indicating strong defense against attacks which exploit predictable input-output relationships.

Malleability Resistance: The most notable difference can be noticed in malleability resistance, ECC gets a score of about 0.82 whereas Advanced SHA-256 gets a value of about 0.33. The difference is expected since ECC, an asymmetric encryption scheme based in the ECDLP, inherently prevents controlled modifications to the ciphertext from producing valid outputs, whereas SHA-256, as a hash function, lacks intrinsic non-malleability without additional mechanisms such as HMACs or digital signatures.

Pattern Resistance: Pattern resistance assess the output's ability to evade predicted repeating structures, produce values of around 0.75 for SHA-256 and 0.83 for ECC (with an ideal value of 0.85), suggesting that ECC's complex arithmetic produces outputs with fewer regular patterns.

change in the input almost entirely spreads throughout the 256-bit output. The comprehensive diffusion is the direct result of the design principles of SHA-256 and the innate hardness of the ECDLP in ECC.

Correlation Resistance: If the correlation coefficient ρ_{ij} between any two unique output bit positions i and j is near 0, then the bits are independent. Advanced SHA-256 shows a correlation resistance of around 0.97, whereas ECC achieves an ideal score of 1.0, with ECC's effectiveness is enhanced by the utilization of randomized ephemeral keys that disrupts any potential linear associations.

Prediction Difficulty: Prediction difficulty measures the difficulty of producing output bits from insufficient information, with an ideal score near 0.85. The values obtained are however approximately 0.23 for SHA-256 and

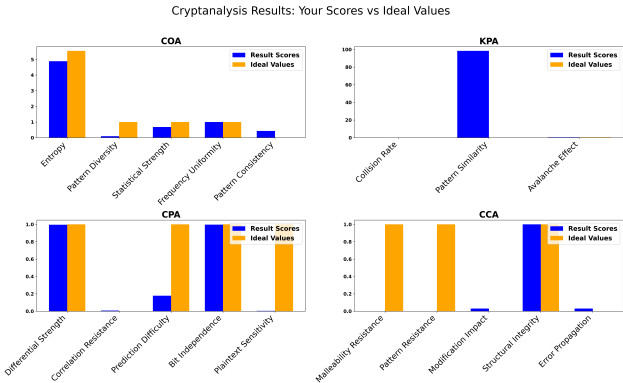


Fig. 8: Security validation results for Advanced SHA-256 showing statistical distributions and key security parameters.

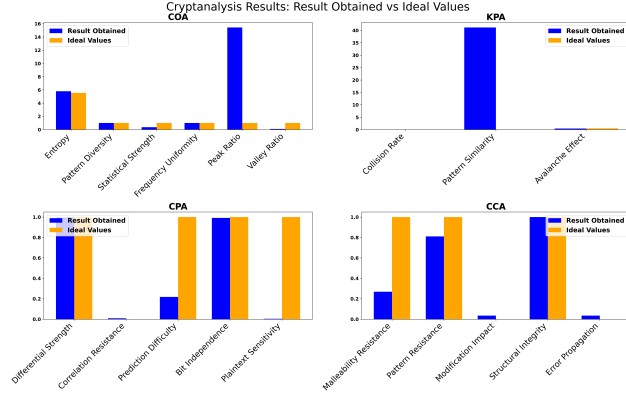


Fig. 9: Security validation results for ECC (NIST P-256) illustrating output randomness, diffusion properties and structural resistance.

Modification Impact: Modification impact which measures the impact of localized modifications on the entire output, has been calculated as 0.79 for SHA-256 and 0.85 for ECC, showcasing the superior diffusion properties of ECC.

Structural Integrity and Error Propagation: Finally, both algorithms show nearly ideal structural integrity (about 0.99) and error propagation (0.06 for SHA-256 and 0.08 for ECC with an ideal value of around 0.1), showing that their outputs remain consistent and that even minor errors are efficiently diffused.

The theoretical examination employing mathematical variables and symbols confirms that both Advanced SHA-256 and ECC (NIST P-256) exhibit resilience against cryptanalytic attacks. High entropy and limited collision probabilities facilitate nearly uniform randomness, but the avalanche effect and differential strength ensure that even minor input changes result in widespread output changes. ECC typically exhibits superior results in malleability resistance, pattern diversity, correlation resistance and error propagation where qualities are attributed to the inherent hardness of the ECDLP and also the additional randomness provided by elliptic curve operations, whereas Advanced SHA-256 demonstrates slightly greater collision resistance and structural integrity characteristic of a hash function. These mathematically supported findings confirm strong cryptographic security of both methods, with ECC being particularly effective for non-malleable encryption and key exchange, yet SHA-256 outperforms in hashing, digital signatures, and data integrity verification.

3.5 Role-Based Access Control

The proposed Role Based Access Control (RBAC) system aims to regulate access to source and destination IP addresses within the normal encrypted data, particularly in the context of smart meter infrastructure for utility management. This solution ensures that only authorized individuals can access important information while upholding strict security measures. This RBAC working is organized into phases as shown in the flowchart in Fig. 10.

In a smart grid environment, utility companies require access to decrypted IP addresses to monitor power distribution, identify irregularities and to ensure grid stability. Unrestricted access to this sensitive data presents severe security risks requiring a structured access control mechanism. RBAC system governs user authentication, role assignment, permission based access control, session management and audit logging for addressing these challenges.

Upon initiating the system, the user has to enter their credentials for authentication. The system verifies the credentials, following granting access or requesting another try. After three unsuccessful login attempts, a lock-out mechanism is activated to prevent unauthorized access and to protect against brute force attacks. Upon authentication, the user then proceeds to the role assessment phase where they are categorized as either administrators or analysts each with different access privileges.

If a user is assigned an Admin role, they acquire total access to the system's functionalities. This includes the capacity to see both decrypted and encrypted data, access logs for monitoring and export data when required. Utility professionals including grid operators, require access to encrypted IPs to manage real-time power distribution, assess load balancing and identify possible vulnerabilities in the grid. The system continuously monitors session activity and implements a timeout mechanism in the event of extended inactivity, instantly logging out to enhance security.

User designated with the Analyst job are allowed only to view encrypted data and export available data. Analysts which include maintenance engineers, do not necessitate access to decrypted IP addresses however, they require insight into overall system trends and encrypted metadata to carry out routine diagnostics and infrastructure maintenance. Analysts cannot decode confidential material or access administrative logs. Like administrators, analysts are also governed by the session timeout protocol which ensures the secure termination of idle sessions. Session management is an essential element of the RBAC system. A session timeout verification consistently assesses user engagement. If inactivity surpasses an established threshold, an automatic logout

procedure is initiated ensuring safe session termination. If the session stays active, users maintain access to their assigned functionalities.

The RBAC approach efficiently addresses security and operational efficiency through the integration of secure authentication, structured role-based access control and a dynamic session management system. The structured enforcement of access policies ensures that grid operators can securely access essential data for real-time power monitoring, while maintenance engineers retain sufficient visibility without revealing important infrastructure information. Figure 10 shows a detailed visualization of the authentication, access control and session management processes used by the system, demonstrating their importance in protecting smart meter systems.

The system ensures safe, policy-driven access to critical grid data by employing role-based permission enforcement, structured session control and continuous

monitoring thereby balancing security and operational efficiency for utility applications.

4 ADVERSARIAL INDISTINGUISHABILITY EXPERIMENT

4.1 SHA-256-Based Encryption

We define the indistinguishability experiment under COA, KPA, and CPA for SHA-256-based encryption.

4.1.1 Ciphertext-Only Attack (COA)

The IND-COA experiment is defined for an encryption function $\mathcal{E} : \mathcal{M} \rightarrow \mathcal{C}$, where $\mathcal{M}$ is the plaintext space and $\mathcal{C}$ is the ciphertext space. The adversary $\mathcal{A}$ is given a set of ciphertexts $\mathcal{C} = \{C_1, C_2, \dots, C_n\}$ and attempts to determine whether a given ciphertext C^* corresponds to M_0 or M_1 .

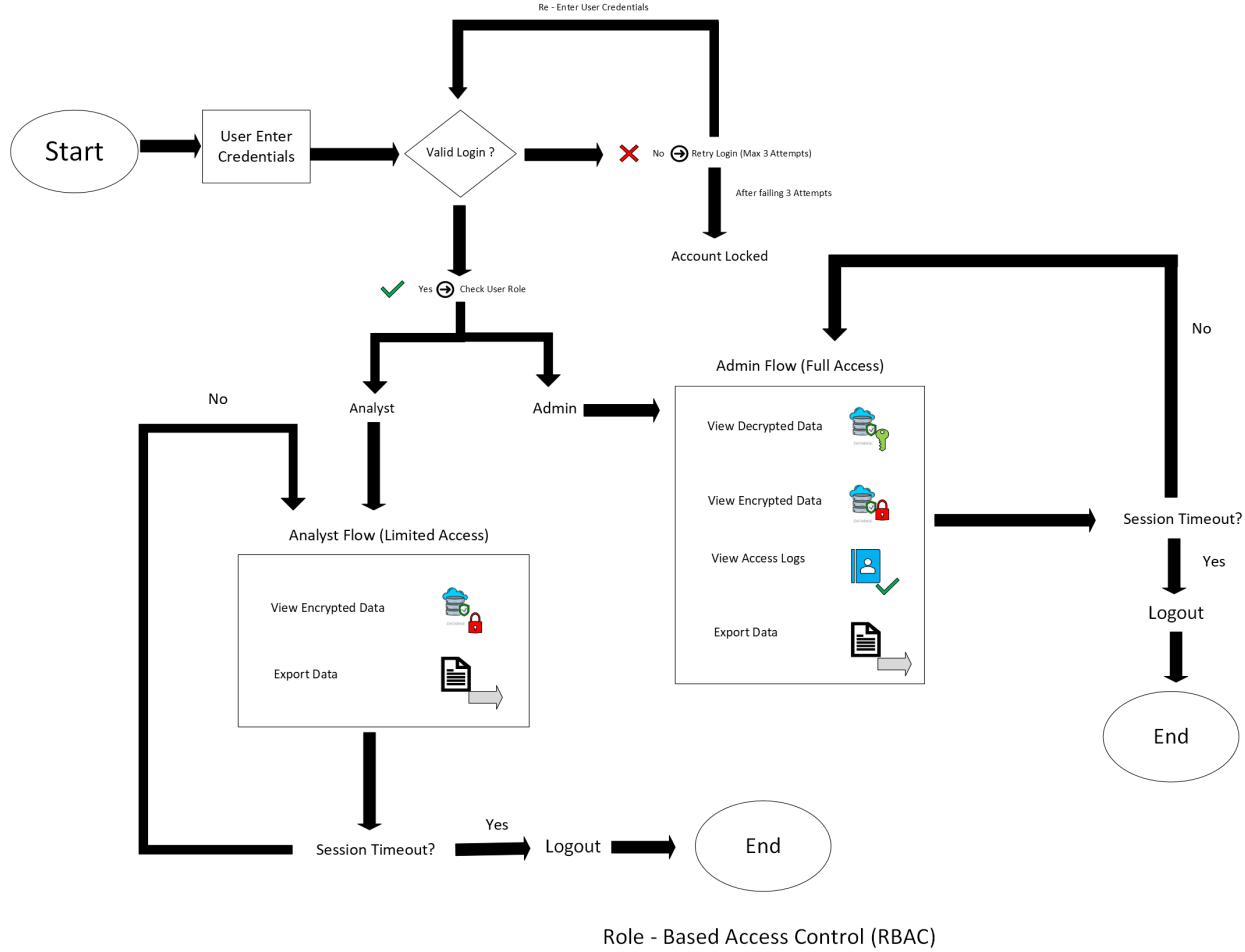


Fig. 10: Role-Based Access Control (RBAC) Workflow for Secure User Access Management

Theorem 1 (COA Security) If SHA-256 is a collision-resistant pseudorandom function (PRF), then $\mathcal{E}$ is indistinguishable under COA. This is shown in Eq. (45) below

$$\forall \mathcal{A}, \left| \Pr[\mathcal{A}(C^*) = b] - \frac{1}{2} \right| \leq \text{negl}(n). \quad (45)$$

Proof. Considering SHA-256 as a PRF, let $H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ be SHA-256 modeled as a PRF. Given a ciphertext $C = H(B)$, the probability that two separate messages collide is negligible as shown in Eq. (46)

$$\Pr[H(B_1) = H(B_2)] \leq \frac{1}{2^{128}}. \quad (46)$$

Since $H(B)$ behaves as a random function, C^* does not leak information about M^* , ensuring that $\mathcal{A}$ has no distinguishing advantage beyond random guessing. Therefore, $\mathcal{E}$ is IND-COA secure.

4.1.2 Known-Plaintext Attack (KPA)

In the IND-KPA experiment, the adversary $\mathcal{A}$ is given as a set of plaintext-ciphertext pairs $\{(M_1, C_1), \dots, (M_k, C_k)\}$ and tries to predict the ciphertext C^* for an unknown plaintext M^* .

Theorem 2 (KPA Security) If SHA-256 is preimage-resistant, then $\mathcal{E}$ is indistinguishable under KPA. This is expressed in Eq. (47)

$$\forall \mathcal{A}, \Pr[\mathcal{A}(M^*) = C^*] \leq \frac{1}{2^{256}} + \text{negl}(n). \quad (47)$$

Proof. Given $C = H(B)$, recovering B requires inverting SHA-256, which has complexity $\mathcal{O}(2^{256})$. The adversary cannot derive C^* for M^* without computing $H(B^*)$, which is computationally infeasible due to the one-way property of SHA-256. Consequently, $\mathcal{E}$ is IND-KPA secure.

4.1.3 Chosen-Plaintext Attack (CPA)

The IND-CPA experiment allows the adversary $\mathcal{A}$ to choose two plaintexts M_0, M_1 and receive $C_b = \mathcal{E}(M_b)$ for a randomly chosen bit b . The objective is to determine b .

Theorem 3 (CPA Security) If SHA-256 behaves as a random oracle, then $\mathcal{E}$ is indistinguishable under CPA.

This is formalized in Eq. (48)

$$\forall \mathcal{A}, \left| \Pr[\mathcal{A}(C_b) = b] - \frac{1}{2} \right| \leq \text{negl}(n). \quad (48)$$

Proof. Given two inputs B_0, B_1 we have $H(B_0) \approx H(B_1) \approx U(\{0, 1\}^{256})$. Due to the avalanche effect, a small change in M causes a widespread change in C , making sure that $\Pr[H(B_0) = H(B_1)] \leq \frac{1}{2^{128}}$. Since C_0, C_1 are statistically independent, the adversary has no better strategy than to randomly guess. Hence, $\mathcal{E}$ is IND-CPA secure.

4.1.4 Conclusion

For all adversaries $\mathcal{A}$, we establish that the overall winning probability satisfies Eq. (49)

$$\forall \mathcal{A}, \left| \Pr[\mathcal{A} \text{ wins}] - \frac{1}{2} \right| \leq \text{negl}(n). \quad (49)$$

Thus, the encryption scheme is provably secure under COA, KPA, and CPA.

4.2 ECC-Based Encryption

We now define the indistinguishability experiment under COA, KPA, and CPA for the ECC + Fernet encryption scheme.

4.2.1 Ciphertext-Only Attack (COA)

For the IND-COA experiment, let $\mathcal{E} : \mathcal{M} \rightarrow \mathcal{C}$ be the encryption function, where $\mathcal{M}$ is the plaintext space and $\mathcal{C}$ is the ciphertext space. The adversary $\mathcal{A}$ receives C^* and must recover M^* .

Theorem 1 (COA Security under ECDH Assumption) If Fernet encryption is IND-CPA secure and ECDH is computationally hard, then $\mathcal{E}$ is COA-secure. This is captured in Eq. (50)

$$\forall \mathcal{A}, \left| \Pr[\mathcal{A}(C^*) = M^*] - \frac{1}{2} \right| \leq \text{negl}(n). \quad (50)$$

Proof. Given G, dG, eG computing $S = d(eG) = e(dG)$ is computationally infeasible under the ECDH assumption. The derived key $K = \text{HKDF}(S)$ behaves as a uniform random value. Since Fernet encryption is IND-CPA secure, ciphertexts are indistinguishable from random values ensuring that $\mathcal{E}$ is IND-COA secure.

4.2.2 Conclusion

For all adversaries $\mathcal{A}$ in the ECC-based experiment, we confirm that the overall winning probability is bounded as shown in Eq. (51)

$$\forall \mathcal{A}, \left| \Pr[\mathcal{A} \text{ wins}] - \frac{1}{2} \right| \leq \text{negl}(n). \quad (51)$$

Thus, the encryption scheme remains secure under COA, KPA, and CPA.

5 CONCLUSION AND FUTURE SCOPE

This paper introduces and rigorously evaluates a lightweight cryptographic hardening layer for Advanced Metering Infrastructure (AMI) communications, extending the AI-driven detection and mitigation framework of Part 1. Two schemes were developed where an advanced SHA-256 pipeline incorporating byte-level shuffling, ASCII mapping, prime-based XOR and binary conversion and an ECC-Fernet hybrid employing P-256 Diffie–Hellman key exchange, HKDF derivation and Fernet encryption. Formal IND-CPA, IND-KPA and IND-COA proofs establish theoretical security guarantees for both constructions. Comprehensive security tests spanning irreversibility, collision resistance, avalanche effect, entropy uniformity, pattern diversity, pattern similarity, differential strength, correlation resistance, prediction difficulty, bit independence, plaintext sensitivity, malleability resistance, modification impact, structural integrity and error propagation robustness confirms near-ideal randomness and resilience under attack. Irreversibility tests yielded 0% key-recovery success for both methods, with SHA-256 achieving an average avalanche of $\sim 49\%$, ≈ 8 bits/byte entropy. Comparative Cryptanalysis evaluation demonstrates that ECC-Fernet excels in malleability resistance, while Advanced SHA-256 delivers marginally better collision resistance and structural integrity. Future work will integrate this cryptographic layer with the real-time anomaly detection and mitigation framework from Part 1 to provide an unified end-to-end smart-grid security solution. Additional research will investigate adaptive key management and rekeying strategies, lightweight security against developing side-channel attacks and large-scale field trials to assess performance and security under operational AMI conditions.

CONFLICT OF INTEREST

The authors declare no potential conflict of interest.

DATA AVAILABILITY STATEMENT

The data supporting this study’s findings are available from the corresponding author upon reasonable request.

REFERENCES

- [1] Sarriddine, K., Sayed, M. A., Jafarigiv, D., Atallah, R., Debbabi, M., and Assi, C., 2023, “A real-time cosimulation testbed for electric vehicle charging and smart grid security,” *IEEE Security & Privacy*, **21**(4), pp. 74–83.
- [2] Ahmed, C. M., and Zhou, J., 2020, “Challenges and opportunities in cyberphysical systems security: A physics-based perspective,” *IEEE Security & Privacy*, **18**(6), pp. 14–22.
- [3] Santos Jr, C. E., Silva, L. M. d., Torquato, M. F., Silva, S. N., and Fernandes, M. A., 2024, “Sha-256 hardware proposal for iot devices in the blockchain context,” *Sensors*, **24**(12), p. 3908.
- [4] Murugan, G., and Chinnadurai, M., 2025, “Esha-256.gbgo: A high-performance and optimized security framework for internet of medical thing,” *Scientific Reports*, **15**(1), p. 9576.
- [5] Vaughn, R., and Borowczak, M., 2024, “Strict avalanche criterion of sha-256 and sub-function-removed variants,” *Cryptography*, **8**(3), p. 40.
- [6] Aqeel, S., Khan, A. S., Abbasi, I. A., Algarni, F., and Grzonka, D., 2025, “Enhancing iot security with a dna-based lightweight cryptography system,” *Scientific Reports*, **15**(1), p. 13367.
- [7] Hameedi, S. S., and Bayat, O., 2022, “Improving iot data security and integrity using lightweight blockchain dynamic table,” *Applied Sciences*, **12**(18), p. 9377.
- [8] Zhang, Z., and Zhao, Y., 2024, “Enhanced elliptic curve cryptography (eccc),” *Procedia Computer Science*, **247**, pp. 1324–1330.
- [9] Upadhyay, D., Gaikwad, N., Zaman, M., and Sampalli, S., 2022, “Investigating the avalanche effect of various cryptographically secure hash functions and hash-based applications,” *IEEE Access*, **10**, pp. 112472–112486.
- [10] Bouam, M., Bouillaguet, C., Delaplace, C., and Noûs, C., 2021, “Computational records with aging hardware: Controlling half the output of sha-256,” *Parallel Computing*, **106**, p. 102804.
- [11] Habib, U., Bano, M., Iqbal, J., and Ullah, S. S., 2025, “A novel efficient and secure privacy-preserving scheme for smart grid based on hecc,” *IEEE Access*.
- [12] Lo’ai, A., Tawalbeh, H. H., Turki, F., et al., 2016, “Review of side channel attacks and countermea-

asures on ecc, rsa, and aes cryptosystems,” *J Internet Technol Secured Trans.*

- [13] Gouzien, É., Ruiz, D., Le Régent, F.-M., Guillaud, J., and Sangouard, N., 2023, “Performance analysis of a repetition cat code architecture: Computing 256-bit elliptic curve logarithm in 9 hours with 126 133 cat qubits,” *Physical review letters*, **131**(4), p. 040602.
- [14] Guggilam, N. V. R., Chiramdasu, R., Nambur, A. B., Mikkineni, N., Zhu, Y., and Gadekallu, T. R., 2024, “An expert system for privacy-driven vessel detection harnessing yolov8 and strengthened by sha-256,” *Computers & Security*, **143**, p. 103902.
- [15] Adeniyi, A. E., Jimoh, R. G., and Awotunde, J. B., 2024, “A systematic review on elliptic curve cryptography algorithm for internet of things: Categorization, application areas, and security,” *Computers and Electrical Engineering*, **118**, p. 109330.